Движок протокола — справочник API

Английский оригинал: PROTOCOL_API.md.

Публичный интерфейс движка протокола и гипотез. Предназначен для модуля GUI и для всех, кто переиспользует анализ, не читая реализацию. Контракт нормализованного захвата определён в `docs/CONTRACT.md` и `docs/schemas/capture.schema.json`; этот модуль его не меняет.

1. Публичный API

Все функции и типы ниже импортируемы из показанных путей пакета. GUI работает с `apply_rule` / `apply_rule_fields`, `build_result`, `verify_on_corpus`, `ResultStore` и `compare_reports`.

Правила

- `src.protocol.rule.parse_rule(raw: dict) -> Rule` — построить `Rule` из разобранного объекта JSON/YAML. Выбрасывает `RuleError` на испорченном правиле.
- `src.protocol.rule.load_rule(path: str) -> Rule` — прочитать JSON или YAML с диска и разобрать.
- `src.protocol.rule.Rule` — dataclass: `schema_version, rule_id, rule_version, name, scope, framing, fields`.
- `.to_dict()`, `.bump_version(**changes) -> Rule` (возвращает копию с `rule_version + 1`), `.direction`, `.applies_to(direction)`, `.is_out_of_date(result_rule_version)`.
- `src.protocol.rule.FieldSpec` — dataclass: `name, offset, type, byte_order, length, enum, hypothesis, expected, missing_ok`.

Фрейминг и применение

- `src.protocol.framing.FramingStrategy.from_dict(spec: dict) -> FramingStrategy` — разобрать блок `framing`. Выбрасывает `FramingError`.
- `src.protocol.framing.frame_stream(stream: bytes | DirectionalStream, strategy: FramingStrategy) -> list[Message]` — разбить поток байт на кандидаты-сообщения. У `Message` есть `offset, length, complete, reason`.
- `src.protocol.engine.apply_rule(stream: DirectionalStream | bytes, rule: Rule, session_id: str = "", direction: str = "A_to_B") -> list[MessageResult]` — сфреймить поток и раскодировать каждое объявленное поле каждого сообщения. Когда область правила исключает направление, каждое сообщение `not_applicable`.
- `src.protocol.engine.apply_rule_fields(...) -> list[FieldResult]` — то же, но плоский список результатов по полям.
- `src.protocol.engine.flatten_fields(results: list[MessageResult]) -> list[FieldResult]`.
- `MessageResult` — `offset, length, status, fields, complete, session_id, direction, reason, bytes_hex`; `.end`, `.field_values()`, `.counterexample()`.
- `FieldResult` — `message_offset, message_length, field_name, field_type, field_offset, field_length, value, status, hypothesis, provenance_range, session_id, direction, reason`.
- `Counterexample` — `status, reason, session_id, direction, message_offset, message_length, field_name, field_offset, field_length, bytes_hex, provenance_range`.

Вход захвата

- `src.protocol.stream.load_capture(path: str) -> Capture` — прочитать нормализованный захват (контрактный формат) с диска.
- `src.protocol.stream.capture_from_dict(raw: dict) -> Capture` — построить из разобранного объекта.
- `Capture.stream(session_id: str, direction: str) -> DirectionalStream`; `Capture.iter_streams() -> list[tuple[str, str, DirectionalStream]]`; `Capture.capture_hash`.
- `src.protocol.stream.DirectionalStream.from_bytes(data: bytes)`, `.from_direction(direction: dict)`, `.holes(start, end)`, `.has_gap(...)`, `.has_ambiguity(...)`, `.provenance`.

Результат

- `src.protocol.result.build_result(rule: Rule, capture_id: str, messages: list[MessageResult]) -> ApplicationResult` — построить контрактный объект результата.
- `ApplicationResult.to_dict()`; `write_result(result, path, validate=True)`; `validate_result(payload, schema_path=None)`; `message_status(result_payload: dict, offset: int) -> Status | None`. `write_result` выбирает формат по суффиксу пути: `.jsonl` пишет по одному компактному сообщению на строку, `.csv` пишет заголовок плюс одну строку на сообщение с колонкой на каждое объявленное поле; любой другой суффикс — контрактный JSON, который проверяется по схеме.

Гипотезы

- `src.hypothesis.corpus.CorpusStream.from_bytes(data, session_id="", direction="A_to_B")` — обернуть сырой поток для проверки на корпусе.
- `src.hypothesis.corpus.verify_on_corpus(rule: Rule, corpus: list[CorpusStream | DirectionalStream]) -> VerificationReport` — применить правило к каждому потоку и подытожить. Отчёт имеет `rule_id, rule_version, totals, total, contradictions, messages, summary`, плюс `.counts()`, `.is_relevant()`, `.is_confirmed()`, `.to_dict()`. `is_confirmed()` истинно только когда правило дошло до хотя бы одного применимого сообщения и ничто ему не противоречило; отчёт, применимый нигде, — не подтверждение. `src.protocol.cli.report_from_dict(payload) -> VerificationReport` пересобирает отчёт из сохранённого JSON-документа `verify`, чтобы `diff`, `metrics` и `alternatives` могли читать отчёт без повторного прогона правила.
- `src.hypothesis.diff.diff_rules(rule_a, rule_b) -> RuleDiff` — добавленные, удалённые и изменённые поля, изменения фрейминга и области. Каждая запись на уровне поля — `FieldChange` (`name`, `kind`, `before`, `after`). `.to_dict()`; `format_rule_diff(diff) -> str`.
- `src.hypothesis.diff.diff_reports(report_a, report_b) -> ReportDiff` — `resolved_counterexamples, introduced_counterexamples, unchanged_mismatches, matched_delta, coverage_delta, version_a, version_b`. `.to_dict()`; `format_report_diff(diff) -> str`.
- `src.hypothesis.metrics.compute_metrics(report) -> RuleMetrics` — см. раздел метрик ниже.
- `src.hypothesis.journal.load_journal(path) -> list[JournalEntry]`, `parse_journal(text)`, `correlate(messages, entries, window_ms=500) -> CorrelationReport` — см. раздел журнала ниже.
- `src.hypothesis.alternatives.suggest_alternatives(rule, messages, correlations=None) -> AlternativesReport`, `analyze_field(rule, messages, field_name, correlations=None) -> FieldAlternatives` — см. раздел альтернатив ниже.
- `src.hypothesis.alternatives.entropy(field_values) -> float`, `periodicity(field_values) -> int | None`, `bit_pattern(field_values, bit_count=4) -> dict`, `delta_correlation(field_values) -> dict` — чтения формы значений; см. раздел альтернатив ниже.

- `src.hypothesis.narrative.render_hypothesis_narrative(investigation) -> str` — записать гипотезы словаря исследования (формы из `src/report/model.py`) как Markdown, заполненный уже присутствующими числами.
- `src.hypothesis.versioning.ResultStore` — `.add(rule, capture_id, payload) -> StoredResult`, `.results_for(rule_id, capture_id=None)`, `.mark_outdated(rule, capture_id=None) -> list[StoredResult]`, `.to_list()`, `.save(path)`.
- `src.hypothesis.versioning.compare_reports(base, new) -> VersionComparison` — `rule_id`, `base_version`, `new_version`, `base_counts`, `new_counts`, `introduced`, `resolved`.
- `src.hypothesis.status.Status` — перечисление всех статусов; `CORE_STATUSES`, `ALL_STATUSES`.

Командная строка

```
python -m src.protocol.cli apply  --rule rule.json --capture capture.json --out result.json
python -m src.protocol.cli verify --rule rule.json --capture capture.json --out report.json
python -m src.protocol.cli diff   --rule-a v1.json --rule-b v2.json --report-a r1.json --report-
b r2.json
python -m src.protocol.cli metrics --report r1.json --out metrics.json
python -m src.protocol.cli correlate --capture capture.json --journal journal.md --out
correlations.json
python -m src.protocol.cli pair   --rule rule.json --capture capture.json --out pairing.json
python -m src.protocol.cli export --rule rule.json --format kaitai --out out/
python -m src.protocol.cli alternatives --report r1.json --corpus capture.json --journal
journal.md --field value --out alternatives.json
```

Каждая команда печатает человекочитаемую сводку в stderr и машиночитаемый документ в stdout или в `--out`, когда задан.

Ошибки

Разбор и загрузка выбрасывают типизированный подкласс `ValueError`, чтобы вызывающий мог отличить плохой вход от внутренней неисправности:

- `src.protocol.rule.RuleError` — `parse_rule` / `load_rule` на испорченном правиле.
- `src.protocol.framing.FramingError` — `FramingStrategy.from_dict` на плохом блоке фрейминга или фрейминг потока, который его не может удовлетворить.
- `src.protocol.stream.CaptureError` — `load_capture` / `capture_from_dict` на захвате не в контрактном формате.
- `src.hypothesis.journal.JournalError` — `load_journal` / `parse_journal` на журнале, который нельзя прочитать.

2. Формат правила

Машиночитаемое определение: `docs/schemas/rule.schema.json`. Правило — JSON или YAML с этими ключами:

- `schema_version` (int, сейчас `1`), `rule_id`, `rule_version`, `name`.
- `scope.direction` — `"A_to_B"` или `"B_to_A"`; отсутствует — значит любое направление.
- `framing` — см. ниже.
- `fields[]` — упорядоченный список полей.

Ключи поля: `name`, `offset`, `type`, `byte_order` (`"big"` по умолчанию или `"little"`), `hypothesis: true` (смысл поля — предположение, а не наблюдение), `expected: [v1, v2]` (допустимые значения; значение вне списка — `mismatched`) и `missing_ok: true` (поле может отсутствовать — часть по-

тока может быть пропуском — без провала сообщения; тогда оно `uncovered`). Остальные ключи зависят от `type`.

Типы полей

Читаются напрямую из раскладки сообщения:

- Числовые `uint8, uint16, uint32, int8, int16, int32` — фиксированная ширина.
- `bytes` — требует `length`; значение — шестнадцатеричная строка в нижнем регистре.
- `enum` — требует отображение `enum` имён на целые значения.
- `bitmask` — диапазон бит внутри одного или нескольких байтов: `bit_offset` (от младшего бита) и `bit_length`. Ширина хранения — `bit_offset + bit_length`, округлённая вверх до целых байт, поэтому диапазон может пересекать границу байта. `json { "name": "version", "offset": 0, "type": "bitmask", "bit_offset": 0, "bit_length": 2 } { "name": "mode", "offset": 0, "type": "bitmask", "bit_offset": 2, "bit_length": 2 }`
- `string` — `length` для строки фиксированной ширины или `terminated: true` для завершённой NUL (терминатор входит в диапазон поля). `encoding` — `ascii` (по умолчанию), `utf-8` или `latin-1`. Хвостовое NUL-дополнение срезается. `json { "name": "tag", "offset": 4, "type": "string", "length": 8, "encoding": "ascii" } { "name": "name", "offset": 12, "type": "string", "terminated": true }`
- `array` — повторяющиеся элементы: `element_type` (числовой тип или `bytes`), `element_length` (для элементов `bytes`) и счётчик, заданный `count` или `count_field` (другим раскодированным полем). Значение — список. `json { "name": "values", "offset": 2, "type": "array", "element_type": "uint16", "count": 5 } { "name": "items", "offset": 4, "type": "array", "element_type": "bytes", "element_length": 2, "count_field": "item_count" }`
- `padding` — `length` байтов, пропускаемых при чтении, но сохранённых в диапазоне сообщения (для выравнивания). Его значение `null`, и он `uncovered`; сообщение из одного дополнения `uncovered`.

Проверяются, а не извлекаются (диапазон сообщения всё равно резервируется):

- `checksum` — `algorithm` (`xor`, `sum`, `crc8`, `crc16`), `length` (по умолчанию ширина алгоритма: 1 байт, 2 для `crc16`) и диапазон байт, который покрывает сумма, заданный `start` и `end` (смещения относительно начала сообщения; `end` по умолчанию — длина сообщения). Сохранённое значение сравнивается с вычисленным; различие — `mismatched`. `src.protocol.checksums.width_for(algorithm) -> int` возвращает эту ширину по умолчанию, а сам помощник публичен, чтобы вызывающий мог задать размер поля, не дублируя таблицу. `json { "name": "crc", "offset": 6, "type": "checksum", "algorithm": "crc16", "length": 2, "start": 0, "end": 6 }`
- `computed` — `expression` описывает значение, выведенное из других полей. Собственные байты поля (`length`, по умолчанию 1) сравниваются с вычисленным значением. Выражения: `sum_of_values` (`fields`), `sum_of_lengths`, `xor` (`fields`, необязательный `value`), `const` (`value`), `add`/`sub`. Операнды ссылаются по имени поля; если любой операнд отсутствует, поле `incomplete`. `json { "name": "payload_length", "offset": 2, "type": "computed", "length": 2, "expression": { "op": "sum_of_lengths", "fields": ["body"] } } { "name": "total", "offset": 4, "type": "computed", "expression": { "op": "sum_of_values", "fields": ["a", "b"] } }`

Условные (присутствуют только когда условие выполнено):

- `conditional` — `condition` это `{ "field": <name>, "op": ..., "value": ... }` с `op` из `eq`, `ne`, `bit_set`, `bit_clear`, `gt`, `lt`; вложенный объект `field` описывает значение, когда условие выполнено. Когда условие не выполнено, значение `null`, и поле `uncovered` (оно не проваливает сообщение). `json { "name": "value_2", "type": "conditional", "condition": { "field":`

`"flags", "op": "bit_set", "value": 1 }, "field": { "name": "value_2", "offset": 6, "type": "uint16" } }`

Ссылки между полями (`conditions`, `computed`, `count_field`) разрешаются по имени против полей, объявленных ранее в списке. Ссылка на необъявленное поле отвергается при разборе правила.

Фрейминг

`framing.type` — одно из `length_prefixed`, `fixed_size`, `marker_based`, `manual`. Только у `length_prefixed` есть неоднозначность в том, что считает значение длины; её разрешает `length_covers`:

`length_covers`	Что считает значение длины	Длина сообщения
`payload` (по умолчанию)	только нагрузку	`length_offset + length_size + value`
`payload_and_length_field`	нагрузку плюс поле длины	`length_offset + value`
`entire_message`	всё сообщение	`value`

Для `length_prefixed` `length_offset` — смещение байта значения длины внутри сообщения, `length_size` — его ширина в байтах, `byte_order` — его порядок байт.

Примеры для одного объявленного значения `6` при `length_offset = 2`, `length_size = 2`:

```
{ "type": "length_prefixed", "length_offset": 2, "length_size": 2, "length_covers": "payload" }
```

общая длина = 2 + 2 + 6 = 10 байт.

```
{ "type": "length_prefixed", "length_offset": 2, "length_size": 2, "length_covers":
"payload_and_length_field" }
```

общая длина = 2 + 6 = 8 байт.

```
{ "type": "length_prefixed", "length_offset": 2, "length_size": 2, "length_covers":
"entire_message" }
```

общая длина = 6 байт.

Устаревший булев `length_includes_payload` всё ещё принимается для старых правил: `true` отображается на `entire_message`, `false` — на `payload`. Новым правилам следует использовать `length_covers`.

`fixed_size` использует `size`. `marker_based` использует `start_bytes` и `end_bytes` (строки байт можно задавать как `"hex:aa55"`, `[0xaa, 0x55]` или простым текстом) и `include_markers`. `manual` использует явный список `messages` из `{offset, length}`.

3. Формат результата

Машиночитаемое определение: `docs/schemas/result.schema.json`. Результат — выход `apply`, потребляемый GUI и модулем отчёта:

```
{
  "contract_version": 1,
  "rule_id": "set_parameter_request",
  "rule_version": 1,
  "capture_id": "sha256:...",
  "messages": [
    { "offset": 0, "length": 8,
      "fields": { "command": 4, "parameter_value": 21 },
      "status": "matched",
      "provenance_range": { "offset": 0, "length": 8, "packets": [7] } }
  ],
  "summary": { "matched": 205, "mismatched": 12, "incomplete": 0,
               "ambiguous": 0, "uncovered": 0, "not_applicable": 0,
               "outdated": 0, "unknown": 0 }
}
```

Статусы

Уровень сообщения:

- `matched` — фрейминг и каждое объявленное поле прочитаны согласованно с правилом.
- `mismatched` — значение поля противоречит правилу (например, значение `expected` не встретилось).
- `incomplete` — сообщение или поле пересекает пропуск или конец потока.
- `ambiguous` — диапазон поля перекрывает байты, которые пересборка сообщила как противоречивые.
- `uncovered` — ни одно объявленное поле не присутствует в этом сообщении.
- `not_applicable` — область правила исключает это направление.

Уровень результата:

- `outdated` — результат произведён более старой версией правила.
- `unknown` — зарезервировано.

`matched` значит «согласовано с правилом на проверенных сообщениях». Это не доказательство; более широкий корпус может правилу противоречить, поэтому и собираются контрпримеры.

4. Разобранный пример

Правило (`rule.json`):

```
{
  "schema_version": 1,
  "rule_id": "set_parameter_request",
  "rule_version": 1,
  "name": "set_parameter_request",
  "scope": { "direction": "A_to_B" },
  "framing": { "type": "length_prefixed", "length_offset": 2, "length_size": 2,
               "byte_order": "big", "length_covers": "entire_message" },
  "fields": [
    { "name": "command", "offset": 0, "type": "uint8", "expected": [4] },
    { "name": "flags", "offset": 1, "type": "uint8" },
    { "name": "payload_length", "offset": 2, "type": "uint16", "byte_order": "big" },
    { "name": "parameter_value", "offset": 4, "type": "uint16", "byte_order": "big",
"hypothesis": true }
  ]
}
```

Захват (`capture.json`, произведённый модулем захвата) держит три 8-байтных сообщения в сессии `s1`, направление `A_to_B`: два с `command = 4` и одно с `command = 9`.

Программное использование:

```
from src.protocol.stream import load_capture
from src.protocol.rule import load_rule
from src.protocol.engine import apply_rule
from src.protocol.result import build_result, write_result

capture = load_capture("capture.json")
rule = load_rule("rule.json")
stream = capture.stream("s1", "A_to_B")
messages = apply_rule(stream, rule, "s1", "A_to_B")
result = build_result(rule, capture.capture_hash, messages)
write_result(result, "result.json")
```

Командная строка:

```
python -m src.protocol.cli apply --rule rule.json --capture capture.json --out result.json
python -m src.protocol.cli verify --rule rule.json --capture capture.json --out report.json
```

Выход `apply` (сокращённо):

```
{
  "contract_version": 1,
  "rule_id": "set_parameter_request",
  "rule_version": 1,
  "capture_id": "sha256:...",
  "messages": [
    { "offset": 0, "length": 8, "fields": { "command": 4, "flags": 0,
      "payload_length": 8, "parameter_value": 21 }, "status": "matched" },
    { "offset": 8, "length": 8, "fields": { "command": 4, "flags": 0,
      "payload_length": 8, "parameter_value": 22 }, "status": "matched" },
    { "offset": 16, "length": 8, "fields": { "command": 9, "flags": 0,
      "payload_length": 8, "parameter_value": 23 }, "status": "mismatched" }
  ],
  "summary": { "matched": 2, "mismatched": 1, "incomplete": 0, "ambiguous": 0,
               "uncovered": 0, "not_applicable": 0, "outdated": 0, "unknown": 0 }
}
```


Выход `verify` перечисляет контрпримеры наряду со счётчиками:


```
{
  "rule_id": "set_parameter_request",
  "rule_version": 1,
  "total": 3,
  "counts": { "matched": 2, "mismatched": 1, "...": 0 },
  "contradictions": [
    { "status": "mismatched", "session_id": "s1", "direction": "A_to_B",
      "message_offset": 16, "message_length": 8,
      "reason": "value 9 not in expected [4]",
      "field_name": "command", "field_offset": 0, "field_length": 1,
      "bytes_hex": "0900000800170000",
      "provenance_range": { "offset": 16, "length": 1, "packets": [7] } }
  ],
  "summary": "2 messages matched, 1 counterexamples (1 mismatched)",
  "capture_id": "sha256:..."
}
```


5. Чтение контрпримеров

`Counterexample` отвечает на вопрос «какие байты расходятся и откуда они».

- `message_offset` и `message_length` задают положение сообщения внутри направленного потока (позиции байт после пересборки).
- `field_name`, `field_offset`, `field_length` задают offending поле относительно начала сообщения.
- `bytes_hex` — сырые байты сообщения. `bytes.fromhex(bytes_hex)` восстанавливает их. В примере сообщение — `09 00 00 08 00 17 00 00`, а поле — его первый байт `09`.
- `provenance_range` привязывает диапазон назад к источнику: `{ "offset": start, "length": n, "packets": [packet_index, ...] }`. `offset` и `length` — позиции в направленном потоке; `packets` — индексы пакетов захвата (как сообщает модуль захвата), внёсших эти байты. Используйте их, чтобы выбрать исходные пакеты и проверить байты по исходному файлу.

Контрпример уровня поля несёт провенанс диапазона поля, а не всего сообщения, поэтому GUI может подсветить ровно те байты, что расходятся.

6. Версии правил

- Правка правила, меняющая фрейминг или чтение полей, создаёт новую версию: `revised = rule.bump_version()` (необязательно `rule.bump_version(fields=...)`).
- Движок никогда не переписывает исходные байты. Применение уточнённого правила к тому же потоку даёт новые результаты; старые неизменны, пока их не помечат.
- `ResultStore.add(rule, capture_id, payload)` хранит результат, помеченный версией правила, которая его произвела.
- `ResultStore.mark_outdated(revised_rule, capture_id)` переключает хранимые результаты, чья версия старше уточнённого правила, в `outdated`: каждый хранимый результат получает `status = "outdated"`, каждое сообщение внутри его нагрузки перепомечается `outdated`, а его сводка обнуляется с `outdated`, равным числу сообщений. Так старая проверка не выдаётся за текущую.
- `compare_reports(base_report, new_report)` возвращает `VersionComparison` с `resolved` контрпримерами (были раньше, теперь ушли) и `introduced` контрпримерами (новые после уточнения). GUI использует это, чтобы показать эффект правки.

7. Диагностики

Контракт захвата сообщает две диагностики, достигающие движка через `Diagnostic.type` на направлении: `gap` и `ambiguity`. Движок превращает их в статусы.

- `gap` (`{type: "gap", offset, length}`) — байты никогда не наблюдались. Они не заполняются нулями. Сообщение или поле, чей диапазон пересекает пропуск, становится `incomplete`.
- `ambiguity` — перекрывающиеся байты расходятся между ретрансмиссиями. Устройство, которое не неоднозначно, неважно; когда неоднозначный диапазон пересекает сообщение или поле, это сообщение становится `ambiguous`, вместо молчаливого выбора одной версии.
- `incomplete` — используется и когда фреймингу нужны байты за концом потока (обрезанный хвост). Сообщение сообщается с его наблюдаемой длиной и `complete = false`.
- `uncovered` — сообщение, где каждое объявленное поле отсутствует по замыслу (`missing_ok`). Оно не несёт интерпретированного значения, поэтому оно ни matched, ни mismatched.

GUI должен показывать `incomplete`, `ambiguous` и `uncovered` отлично от `mismatched`: они описывают ограничение данных или охвата правила, а не противоречие утверждению.

8. Метрики

`compute_metrics(report)` превращает отчёт проверки в числа, а команда CLI `metrics` пишет их как JSON. `RuleMetrics`:

- `total_messages`, `applicable_messages` (сообщения, которые покрывает область правила), `sessions`, `matched`, `mismatched`, `counterexamples`;
- `coverage = (total - not_applicable) / total` — какую долю корпуса правило достигает;
- `precision = matched / (matched + mismatched)` — из сообщений, которые оно интерпретирует, сколько с ним согласны;
- `fragmentation = total_messages / sessions` — среднее число сообщений на сессию, признак того, что фрейминг действительно разбил поток;
- `unknown_ratio = unknown / total`;

- `uncovered_ratio` — байты потоков, не потреблённые ни одним сфреймленным сообщением, `(stream_bytes - covered_bytes) / stream_bytes`;
- `counterexample_density = counterexamples / applicable * 100`;
- `length_histogram` — длина сообщения в счётчик;
- `type_histogram` — значения поля `command` в счётчик, когда такое поле извлекается.

```
python -m src.protocol.cli metrics --report report.json --out metrics.json
```

9. Корреляция с журналом

Журнал — собственная запись исследователя о том, что происходило во время захвата, записанная простым текстом отдельно от байтов. Каждая не-комментарийная строка — либо `<timestamp> | <action> | key=value | ...`, либо позиционная `<timestamp> | <action> | <parameter> | <result>`; их различают по тому, содержит ли третье поле `=` (позиционная форма хранит `parameter` и `result` как ключи). Временная метка — секунды эпохи или ISO-8601; `#` начинает комментарий, строка, не начинающаяся с цифры, — проза, а пустые строки игнорируются. `parse_timestamp(text) -> float` — разборщик за этим правилом.

```
100.02 | set_temperature | value=21
100.31 | toggle_power
100.40 | read | temperature | 1595
```

`correlate(messages, entries, window_ms=500)` связывает каждое сообщение с временной меткой с ближайшей записью журнала, чья метка в пределах окна. Метка берётся из провенанса сообщения (самое раннее время пакета), поэтому движок записывает `ts` на каждом `FieldResult.provenance_range` и на `MessageResult.timestamp`. Результат — `CorrelationReport`:

- `correlated` — по одной `Correlation` на связанное сообщение: смещение и время сообщения, запись, `delta_ms`, каждое раскодированное значение поля (`message_values`), ключи журнала, совпадающие с именем поля, и `value_agreements`/`value_conflicts` (раскодированное значение против параметра журнала). Конфликт — доказательство, что поле не значит то, что говорит журнал, и он сохраняется, а не отбрасывается.
- `messages_without_timestamp`, `messages_without_entry` — сообщения, которые не удалось связать, и почему;
- `entries_without_message` — строки журнала, за которыми не последовало сообщение.

Корреляция — временное предположение, а не доказательство: близкое сообщение с нужным значением поддерживает гипотезу о поле; пустое окно сообщает, что доказательств не было. Запись используется не более одного раза.

```
python -m src.protocol.cli correlate --rule rule.json --capture capture.json \
    --journal journal.md --window-ms 500 --out correlations.json
```

Связывание запроса и ответа

Корреляция с журналом связывает сообщение с собственными заметками исследователя. Связывание связывает сообщение с другим сообщением: когда правило фреймит оба направления, `build_turns(messages, session_id="", window_seconds=5.0)` упорядочивает сообщения по времени и связывает каждое со следующим сообщением в противоположном направлении. Это в `src/hypothesis/pairing.py`.

Каждый `Turn` держит запрос, ответ (или `None`), наблюдаемый `gap_seconds`, `confidence` в `[0, 1]` и `basis` — список наблюдений, на которых держится уверенность. Сообщение без последователя

в противоположном направлении сообщается как без ответа с уверенностью `0`. Уверенность — фиксированная взвешенная сумма: база `0.40`, противоположные направления `0.15`, наблюдаемый порядок времени `0.15`, ответ в пределах окна `0.10`, оба сообщения сфреймлены чисто `0.10`, оба matched `0.05`, минус `0.20`, когда любое сообщение сфреймлено слабо (`ambiguous`, `incomplete`, `unknown`), и `0.15`, когда метка времени отсутствует. Веса объявлены в начале модуля.

Число — балл по этим наблюдениям, а не вероятность того, что ответ принадлежит запросу. Связывание никогда не утверждает принадлежность; оно сообщает порядок, направление и тайминг и оставляет смысл исследователю.

```
python -m src.protocol.cli pair --rule rule.json --capture capture.json \
    --session s1 --window-ms 500 --out pairing.json
```

`build_turns_for_corpus(streams)` связывает внутри каждой сессии отдельно, поэтому ответ из одного соединения никогда не сопоставляется с запросом из другого.

10. Альтернативы и выгрузка

`analyze_field(rule, messages, field_name, correlations=None)` проверяет конкурирующие чтения одного поля и оценивает каждое по тому, скольким сообщениям оно соответствует. Кандидаты — `constant`, `counter`, `message_length`, `remaining_length`, `checksum_<algo>_<range>` по телу сообщения, `low_cardinality`, `alias_of_<other>` когда поле зеркалит другое, `journal_<key>` когда предоставлены корреляции, и четыре чтения кодировки для числовых полей: `endianness` (счётчик читается как little-endian, тогда как правило говорит big-endian), `offset_shift_+1`/`offset_shift_-1` (соседний байт — настоящее константное поле), `xor_mask_<k>` (значения маскируются константой) и `delta_encoding` (поле хранит шаг с предыдущего сообщения). Чтения кодировки предлагаются только когда объявленные big-endian значения не уже самая простая последовательность, поэтому простой счётчик не сообщается как маскированный. Каждая `Alternative` несёт `support`, `contradict`, `score = support / (support + contradict)` и до нескольких записей `evidence`. `suggest_alternatives` гоняет это по каждому полю, помеченному `hypothesis: true`. Объявленный смысл — один кандидат среди них и может проиграть; выход — это доказательство, а не вердикт.

Четыре дальнейших чтения описывают, как числовое поле меняется, а не что оно значит; они используют только стандартную библиотеку и берут только значения поля. Каждое также предлагается как кандидат, когда его условие выполнено:

- `entropy(field_values) -> float` — энтропия Шеннона в битах на значение. Больше 6 бит намекает на nonce, хэш или случайное поле; от 3 до 6 бит — на значение, которое меняется (счётчик или измерение); меньше 2 бит — на enum или флаги. Имена кандидатов: `entropy_random`, `entropy_parameter`, `entropy_enum`.
- `periodicity(field_values) -> int | None` — наименьший лаг, на котором автокорреляция — ясный локальный пик, или `None`. Монотонный ramp или простой счётчик возвращают `None`, потому что они коррелируют на каждом коротком лаге. Имя кандидата: `periodicity`.
- `bit_pattern(field_values, bit_count=4) -> dict` — насколько устойчивы старшие биты (`bit_count` из `(2, 4, 6)`): `verdict` — `flags_high` выше 90 процентов устойчивости, `contradict` на 50 процентах или ниже, иначе `inconclusive`. Имена кандидатов: `bit_pattern_2`, `bit_pattern_4`, `bit_pattern_6`.
- `delta_correlation(field_values) -> dict` — коэффициент вариации первых разностей: `delta` ниже 0.1 (фиксированный шаг или delta-кодирование), `contradict` выше 0.5. Имя кандидата: `delta_correlation`.

Нечисловые значения вне области для всех четырёх и игнорируются.

```
python -m src.protocol.cli alternatives --rule rule.json --report report.json \
    --journal journal.md --field value --out alternatives.json
python -m src.protocol.cli alternatives --rule rule.json --report report.json \
    --all --out alternatives.json
```

`--field NAME` анализирует одно поле; `--all` анализирует каждое поле-гипотезу в правиле.

`export_kaitai(rule)` возвращает документ Kaitai Struct `.ksy`, а `export_python(rule)` — автономный модуль Python без зависимости от этого проекта. Оба воспроизводят фрейминг и чтение полей; `parse_stream(data)` сгенерированного модуля возвращает те же диапазоны и значения, что и движок, и сам проверяет контрольные суммы и вычисленные поля. Проверка гипотез, неоднозначность и провенанс остаются в движке.

```
python -m src.protocol.cli export --rule rule.json --format kaitai --out out/
python -m src.protocol.cli export --rule rule.json --format python --out parser.py
```

11. Стыковка с GUI

Окно в `src.ui` — тонкий потребитель этого модуля. Оно никогда не разбирает захват и никогда не переизобретает фрейминг: оно загружает нормализованный захват, передаёт направление и правило в `apply_rule` и отрисовывает результат.

Что GUI ожидает

- Нормализованный захват, либо файл (`load_capture`), либо разобранный JSON (`capture_from_dict`). Эталонные выгрузки в `tests/corpus/reference_export/` — ровно этой формы.
- Правило, либо файл (`load_rule`), либо отредактированный текст (`load_rule_text` + `parse_rule`).

Что GUI вызывает

- `CaptureModel.from_file(path)` оборачивает `capture_from_dict` и берёт список сессий и диагностики из `sessions` и `directions[*].diagnostics`.
- `RuleModel.apply` вызывает `apply_rule(stream, rule, session_id, direction)` и держит список `MessageResult`. `counts` и `counterexamples` читаются из него.
- `verify_on_corpus(rule, streams)` поддерживает команду «применить ко всему захвату»; `CorpusStream.from_bytes(data, session_id, direction)` строит её вход.
- Провенанс байта для строки состояния берётся из `DirectionalStream.provenance` (каждый `Hole` несёт `packet_index`, `seq`, `ts`) или из `FieldResult.provenance_range` для поля под указателем.

Детали уровня поля, на которые опирается GUI

- `FieldResult.field_offset` и `field_length` задают диапазон байт внутри его сообщения; hex-просмотр подсвечивает `message.offset + field_offset` и далее.
- `FieldResult.hypothesis` помечает предположение; таблица сообщений его подписывает.
- `MessageResult.status` и `bytes_hex` ведут строку сообщения и список контрпримеров.
- `MessageResult.provenance_range` — не поле сообщения; провенанс читается из полей сообщения (`fields[0].provenance_range`, когда у сообщения есть поля). Именно это использует `src.ui.validation_view` и интеграционные тесты.

Пример

Открыть эталонный захват, применить правило v1 и прочитать результат:

```
from src.protocol.engine import apply_rule
from src.protocol.rule import load_rule
from src.protocol.stream import load_capture

capture = load_capture("tests/corpus/reference_export/corpus_capture_01.normalized.json")
rule = load_rule("examples/corpus_rule_v1.json")
stream = capture.stream("s1", "A_to_B")
messages = apply_rule(stream, rule, "s1", "A_to_B")
matched = sum(m.status.value == "matched" for m in messages)
```

Чтобы произвести контрактный результат, который используют отчёт и выгрузка:

```
from src.protocol.result import build_result

payload = build_result(rule, capture.capture_hash, messages).to_dict()
```

Результат проходит проверку по `docs/schemas/result.schema.json`. Байт, по которому пользователь кликнул, сообщается как `session + direction + offset + packet_index + seq + ts`, чего достаточно, чтобы найти его в исходном захвате.

12. Ограничения

Модель правил плоская по замыслу: массивы повторяют один элемент фиксированной ширины, а `computed`/`conditional` ссылаются на другие объявленные поля, но нет вложенных подзаписей, нет элементов переменной длины и нет раскладки, меняющейся целиком по варианту. Вариант протокола описывается отдельным правилом, ограниченным им. Всё, что требует скриптования по каждому сообщению, принадлежит вызывающему, читающему выход `apply_rule`. Поля выровнены по байтам, кроме `bitmask`. Зашифрованные или сжатые нагрузки не декодируются. Это соответствует заданию, где цель — проверяемая интерпретация потока, а не полное описание протокола; их можно расширить, не меняя контракт результата.